

24 PIN VCU SDK USER MANUAL v1.0.0

SLOKI Software Technologies

Document Number: Rev.
1.0.0

SEP 2026



Contents

Contents	Error! Bookmark not defined.
1. Introduction.....	5
2. Getting Started	5
System Requirements	5
Installation	5
3. Function.....	5
3.1 Output Pin Functions.....	5
3.2 Input Pin Functions.....	6
3.3 CAN RX Functions.....	7
3.4 CAN TX Functions.....	11
3.5 ADC Input Functions	12
4. Enums and Data Structures.....	13
4.1 GPOutput_Signal_En_t.....	13
4.2 GPInput_Signal_En_t.....	15
4.3 AdcInput_En_t.....	15
5. Task Scheduler:.....	17
6. BUILD PROCEDURE	18
6.1 How to find Hex file	19
7. CAN Baud Rate DID Configuration.....	21
7.1 Write CAN Baud Rate (UDS Service 0x2E)	21
7.2 Read CAN Baud Rate (UDS Service 0x22).....	22
8. NVM Read and Write Functions	23
8.1 nvm_status_En_t.....	23
Declaration	23
Enumeration Definition	23
8.2 NVM Write Function.....	24
Function prototype:.....	24
Parameters:	24
Return Value:.....	24

24 PIN VCU MANUAL

Description:.....	24
8.3 NVM Read Function.....	25
Function prototype:	25
Parameters:	25
Description:.....	25

Document Revision.

Version	Description of Changes	Date	Author	Reviewed by	Approved by
V1.0.0	Initial Revision.				

1. Introduction

Welcome to the SDK Manual for 24 pin VCU. This manual provides comprehensive instructions on using the SDK to control output pins, read input pins, and manage CAN (Controller Area Network) communication, including receiving (RX) and transmitting (TX) messages.

2. Getting Started

System Requirements

- Supported VCU: 24 pins
- Development Environment: Visual studio code or any development IDE
- Programming Language: C

Installation

1. Download the SDK.
2. Unzip the file and copy the contents to your project directory.
3. Include the SDK header files in your project.

3. Function

3.1 Output Pin Functions

Sets the specified pin to the given value (HIGH or LOW).

Function prototype:

Set_GPO_State (GPOOutput_Signal_En_t GPOOutput_Channel_Num, bool PinState_b);

Include file: [databank.h](#)

Parameters:

GPOOutput_Channel_Num: pin number to set.

PinState_b: GPIO_PIN_HIGH to set the pin HIGH

: GPIO_PIN_LOW to set the pin LOW

Return type: void

NOTE: Please refer Enums and structure section for Enum explanations. Example:

```
#include "databank.h" void

outputFunction()

{

    Set_GPO_State (A7_GPOUTPUT1_HS1_E, GPIO_PIN_LOW); //set the the 1 pin to low

    Set_GPO_State (B7_GPOUTPUT2_HS2_E, GPIO_PIN_HIGH); //setthethe2 pin
    to high

    return;

}
```

3.2 Input Pin Functions

Reads the state of the specified input pin.

Function prototype: Get_GPInput_State(**GPInput_Signal_En_t**

GPI_Channel_Num) **Parameters:**

GPI_Channel_Num : The pin number to read. Return

type: uint8_t (8bit)

- true if the pin is HIGH, false if the pin is LOW.

NOTE: Please refer Enums and structure section for Enum explanations.

Example:

```
#include " databank.h "
```

```
void inputFunction()
{
    uint8_t pinState;

    pinState=Get_GPInput_State(B6_GPINUPUT1_E); if (pinState)
    {
        // Pin 1 is HIGH
    }
    else
    {
        // Pin 1 is LOW
    }
}
```

3.3 CAN RX Functions

When receiving a CAN message, the following Step must be followed:

Step 1: In the given SDK, there is a PAL folder. Inside the PAL folder, there is a pal_can folder. The pal_can folder contains two files: pal_can_conf.c and pal_can_conf.h.

In pal_can_conf.c you have to configure some data

Step 2: In pal_can_conf.c, there are three macros defined as follows:

```
#define TOTAL_RX_FILTER_CAN_0      (number of receiving message)
#define TOTAL_RX_FILTER_CAN_1      (number of receiving message)
#define TOTAL_RX_FILTER_CAN_2      (number of receiving message)
```

Explanation:

- **TOTAL_RX_FILTER_CAN_0:** This macro defines the total number of CAN messages that CAN0 is configured to receive. Replace (number of receiving messages) with the actual number of messages CAN0 will handle.

-
- **TOTAL_RX_FILTER_CAN_1:** This macro defines the total number of CAN messages that CAN1 is configured to receive. Replace (number of receiving messages) with the actual number of messages CAN1 will handle.
 - **TOTAL_RX_FILTER_CAN_2:** This macro defines the total number of CAN messages that CAN2 is configured to receive. Replace (number of receiving messages) with the actual number of messages CAN2 will handle.

Each macro indicates the number of CAN messages you are receiving.

NOTE: Assign the exact number of CAN messages you are receiving to each macro.

Step 3:

In `pal_can_conf.c`, there are three const variables defined as follows:

```
const uint8_t Can0_Rx_filter_u8 = (number of receiving messages); const uint8_t  
Can1_Rx_filter_u8 = (number of receiving messages); const uint8_t Can2_Rx_filter_u8 =  
(number of receiving messages);
```

These variables also need to be assigned the number of CAN messages you are receiving **NOTE:**

Assign the exact number of CAN messages you are receiving to each macro. Explanation:

- **Can0_Rx_filter_u8:** This constant variable specifies the total number of CAN messages that CAN0 is configured to receive. Replace (number of receiving messages) with the actual number of messages CAN0 will handle.
- **Can1_Rx_filter_u8:** This constant variable specifies the total number of CAN messages that CAN1 is configured to receive. Replace (number of receiving messages) with the actual number of messages CAN1 will handle.
- **Can2_Rx_filter_u8:** This constant variable specifies the total number of CAN messages that CAN2 is configured to receive. Replace (number of receiving messages) with the actual number of messages CAN2 will handle.

These constant variables are used to configure the CAN message filtering by specifying how many messages each CAN channel should expect to receive. This configuration ensures that the system can properly handle and process incoming CAN messages for each channel.

Step 4:

In pal_can_conf.c file

In the provided SDK, the CAN receive filter configurations for CAN0, CAN1, and CAN2 are defined using arrays of the CanRxFilterConf_St_t structure. Each entry in these arrays specifies the filtering criteria for incoming CAN messages.

CAN0 Receive Filter Configuration

The array Can0RxFilterConf_aSt is defined to filter CAN messages for CAN0. The size of the array is determined by the macro TOTAL_RX_FILTER_CAN_0.

```
CanRxFilterConf_St_t Can0RxFilterConf_aSt[TOTAL_RX_FILTER_CAN_0] =
{
    {0x100, 0x7FF, RX_FIFO_BUFF_E, STD_IDE_E, NULL},
    {0x7F0, 0x7FF, RX_FIFO_BUFF_E, STD_IDE_E, NULL},
    {0x7DF, 0x7FF, RX_FIFO_BUFF_E, STD_IDE_E, NULL},
};
```

CAN1 Receive Filter Configuration

The array Can1RxFilterConf_aSt is defined to filter CAN messages for CAN1. The size of the array is determined by the macro TOTAL_RX_FILTER_CAN_1.

```
CanRxFilterConf_St_t Can1RxFilterConf_aSt [TOTAL_RX_FILTER_CAN_1] =
{
    {0x200, 0x7FF, RX_FIFO_BUFF_E, STD_IDE_E, NULL},
    {0x7F0, 0x7F1, RX_FIFO_BUFF_E, STD_IDE_E, NULL},
    {0x7DF, 0x7FF, RX_FIFO_BUFF_E, STD_IDE_E, NULL},
    {0x220, 0x7FF, RX_FIFO_BUFF_E, STD_IDE_E, Call_Back_CAN1_RX_Test},
};
```

CAN2 Receive Filter Configuration

The array Can2RxFilterConf_aSt is defined to filter CAN messages for CAN2. The size of the array is determined by the macro TOTAL_RX_FILTER_CAN_2.

```
CanRxFilterConf_St_t Can2RxFilterConf_aSt[TOTAL_RX_FILTER_CAN_2] =
```

```
{
    {0x103,0x7FF,RX_FIFO_BUFF_E,STD_IDE_E,NULL},
}
```

Explanation

- **Filter ID and Mask:** The first two fields (0x18FEBF0B and 0x1FFFFFFF) (0x100 and 0x7FF) represent the filter ID and mask. The ID specifies the target message ID, and the mask is used to determine which bits of the incoming message ID should be compared against the filter ID.
- **Buffer:** RX_FIFO_BUFF_E specifies the buffer where the filtered message will be stored.
- **IDE:** EXT_IDE_E and STD_IDE_E specify whether the filter is for extended (29-bit) or standard (11-bit) identifiers.
- **Callback:** NULL or function pointers like CAN0_Tx_Test specify the callback function to be called when a message matching the filter is received.

CAN Receive Function Prototype

The function used for handling received CAN messages can have any name, and CAN0_Tx_Test is just an example. The prototype of this function is as follows:

```
void CAN0_Tx_Test (uint8_t CanNum_u8, uint8_t *DataBuff_pu8, uint8_t Dlc_u8, uint8_t CanMode_u8, uint8_t Ide_u8, uint32_t CanId_u32);
```

- **Function Name:** CAN0_Tx_Test (This is an example name; any suitable name can be used)

Arguments

- **CanNum_u8:** The CAN channel number.
- **DataBuff_pu8:** Pointer to the data buffer containing the received message.
- **Dlc_u8:** Data length code of the received message.
- **CanMode_u8:** CAN mode.
- **Ide_u8:** Identifier extension bit (standard or extended identifier).
- **CanId_u32:** CAN identifier of the received message.

Description

The function CAN0_Tx_Test (or any chosen name) is designed to process CAN messages that match specific filtering criteria. It is called when a matching CAN message is received.

Declaration

This function should be declared as extern in one of the header files. To ensure it is available for use in the pal_can_conf.c file, include the appropriate header file.

Header File Inclusion

To use the CAN0_Tx_Test function (or any chosen name) in the pal_can_conf.c file, include the header file where it is declared:

```
#include "your_header_file.h"      // Replace with the actual header file name
```

Example

Here's how you might declare and use the CAN0_Tx_Test function (or any chosen name):

1. **Declaration in Header File** (e.g., your_header_file.h):

```
extern void CAN0_Tx_Test(uint8_t CanNum_u8, uint8_t *DataBuff_pu8, uint8_t Dlc_u8, uint8_t CanMode_u8, uint8_t Ide_u8, uint32_t CanId_u32);
```

2. **Inclusion in pal_can_conf.c:**

```
#include "your_header_file.h"
```

By following this structure, you ensure that the function is properly declared and included, making it available for handling received CAN messages.

3.4 CAN TX Functions

uint8_t CanMsgTransmit (uint8_t CANNuM_u8, uint8_t* DataBuff_pu8, uint8_t Dlc_u8, uint8_t CanMode_u8, uint8_t Ide_u8, uint32_t CanId_u32)

Sends a CAN message.

Parameters:

CanNum_u8 : CAN number

CAN0

CAN1

CAN2

DataBuff_pu8: pointer to the data buffer. Dlc_u8:

Length of the CAN message.

CanMode_u8: 0 - Classic CAN

1 - CAN FD

Ide_u8: 0 - 11bit IDE.

1 - 29-bit IDE.

CanId_u32: message identifier.

Example:

```
void transmittCAN_msg()
{
    uint8_t canNumber = CAN1;
    uint8_t canData_u8[8]={1,2,3,4,5,6,7,8}; uint8_t msgLength_u8
    = 8;
    uint8_t canMode = 0; uint8_t
    canFormat=0;
    CanMsgTransmit(canNumber, canData_u8, msgLength_u8, canMode, canFormat,0x100);
}
```

3.5 ADC Input Functions

Reads the Value of the specified ADC pin.

Function prototype:

Get_ADC_Result(AdcInput_En_t ADC_Channel_Num);

Include file: [databank.h](#)

Parameters:

AdcInput_En_t : The pin number to read.

Return type: uint16_t

NOTE: Please refer Enums and structure section for Enum explanations.

```
#include " databank.h "

uint16_t readADCFunction(void)
{
    uint16_t ADCvalue_u16;

    ADCvalue_u16 = Get_ADC_Result(A4_ADC_INPUT_E);

    return ADCvalue_u16;
}
```

4. Enums and Data Structures

4.1 GPOutput_Signal_En_t

This enumeration defines a set of constants that represent different General Purpose Output (GPO) signals.

Declaration

The GPOutput_Signal_En_t enumeration is declared in the databank.h file.

Enumeration Definition

```
typedef enum
{
    GPOUTPUT_START_E = 0,           // Start of the enumeration
    A7_GPOUTPUT1_HS1_E = GPOUTPUT_START_E,    // highside
    B7_GPOUTPUT2_HS2_E,             // highside
    A8_GPOUTPUT3_HS3_E,             // highside
    B8_GPOUTPUT4_HS4_E,             // highside
    A4_GPOUTPUT5_LS1_E,             // lowside
    A5_GPOUTPUT6_LS2_E,             // low side
    A6_GPOUTPUT7_LS3_E,             // low side
    GPOUTPUT_TOTAL_E,               // Total number of GPO signals
} GPOutput_Signal_En_t;
```

Description

- **GPOUTPUT_START_E = 0:** This is the starting value of the enumeration. The value 0 is assigned to GPOUTPUT_START_E.
- **A7_GPOUTPUT1_HS1_E = GPOUTPUT_START_E:** This assigns A7_GPOUTPUT1_HS1_E the same value as GPOUTPUT_START_E, which is 0. This is a high-side pin associated with the A7 pin at the harness.
- **B7_GPOUTPUT2_HS2_E:** This represents the next value in the enumeration, which will be 1 (as it follows A7_GPOUTPUT1_HS1_E). B7 indicates the pin in the connector or harness, and high side indicates it is a high-side output pin.
- **A8_GPOUTPUT3_HS3_E:** This represents the next value in the enumeration, which will be 2. A8 indicates the pin in the connector or harness, and high side indicates it is a high-side output pin.
- **B8_GPOUTPUT4_HS4_E:** This represents the next value in the enumeration, which will be 3. B8 indicates the pin in the connector or harness, and high side indicates it is a high-side output pin.
- **A4_GPOUTPUT5_LS1_E:** This represents the next value in the enumeration, which will be 4. A4 indicates the pin in the connector or harness, and low side indicates it is a low-side output pin.
- **A5_GPOUTPUT6_LS2_E:** This represents the next value in the enumeration, which will be 5. A5 indicates the pin in the connector or harness, and low side indicates it is a low-side output pin.
- **A6_GPOUTPUT7_LS3_E:** This represents the next value in the enumeration, which will be 6. A6 indicates the pin in the connector or harness, and low side indicates it is a low-side output pin.
- **GPOUTPUT_TOTAL_E:** This represents the total number of GPO signals defined in the enumeration. It will have value 7, which is one more than the last defined GPOUTPUT value (A6_GPOUTPUT7_LS3_E).

4.2 GPIInput_Signal_En_t

This enumeration defines a set of constants that represent different General-Purpose Input (GPI) signals.

Declaration

The GPIInput_Signal_En_t enumeration is declared in the databank.h file.

Enumeration Definition

typedef enum

```
{
    GPINUPUT_START_E    = 0,

    B6_GPINUPUT1_E =    GPINUPUT_START_E,    // B6

    GPINUPUT_TOTAL_E                //Total number of GPI signals
} GPIInput_Signal_En_t;
```

Description

- **GPINUPUT_START_E = 0:** This is the starting value of the enumeration. The value 0 is assigned to GPINUPUT_START_E.
- **B6_GPINUPUT1_E = GPINUPUT_START_E:** This assigns B6_GPINUPUT1_E the same value as GPINUPUT_START_E, which is 0. This input signal is associated with the B6 pin in the connector or harness.
- **GPINUPUT_TOTAL_E:** This represents the total number of GPI signals defined in the enumeration. It will have the value 1, which is one more than the last defined GPINUPUT value (B6_GPINUPUT1_E).

4.3 AdcInput_En_t

This enumeration defines a set of constants that represent different Analog to Digital Converter (ADC) signals.

Declaration

The AdcInput_En_t enumeration is declared in the databank.h file.

Enumeration Definition

typedef enum

```
{
    ADC_START_E = 0,
    C7_ADC1_E = ADC_START_E,
    C4_ADC2_E,
    B4_ADC3_E,
```

```
C1_ADC4_E,  
C5_ADC5_E,  
B5_ADC6_E,  
ADC_TOTAL_E  
} AdcInput_En_t;
```

Description

- **ADC_START_E = 0:** This is the starting value of the enumeration. The value 0 is assigned to ADC_START_E.
- **C7_ADC1_E = ADC_START_E:** This assigns C7_ADC_INPUT_E the same value as ADC_START_E, which is 0. This input signal is associated with the C7 pin in the connector or harness.
- **C4_ADC2_E:** This represents the 1 value in the enumeration, with an assigned value of 1. The input signal corresponds to the C4 pin in the connector or harness.
- **B4_ADC3_E:** This represents the 2 value in the enumeration, with an assigned value of 2. The input signal corresponds to the B4 pin in the connector or harness.
- **C1_ADC4_E:** This represents the 3 value in the enumeration, with an assigned value of 3. The input signal corresponds to the C1 pin in the connector or harness.
- **C5_ADC5_E:** This represents the 4 value in the enumeration, with an assigned value of 4. The input signal corresponds to the C5 pin in the connector or harness.
- **B5_ADC6_E:** This represents the 5 value in the enumeration, with an assigned value of 5. The input signal corresponds to the B5 pin in the connector or harness.
- **ADC_TOTAL_E:** This represents the total number of ADC inputs defined in the enumeration. It will have the value 6, which is one more than the last defined ADC input value (B5_ADC6_E)

5. Task Scheduler:

Task_scheduler_conf.c and Task_scheduler_conf.h

- In this file user can configure the function that needs to be called periodically with the provided periodicity in milli seconds.
- This file contains TaskConf_St structure in which Function and periodicity of that function is configured.

```
const uint8_t Total_task_u8=TOTAL_TASK;
TaskConf_St_t TaskConf_St[TOTAL_TASK] =
{
    {Test_Adc      ,TS_PERIODICTY(50), INIT_VALUE},
    {Test_GpioInput ,TS_PERIODICTY(50), INIT_VALUE},
    {Test_GpioOutput ,TS_PERIODICTY(50), INIT_VALUE},
}
```

- If new function is added to structure configuration table, TOTAL_TASK Enum has to be updated in Task_scheduler_conf.h.
- **Note: New functions should be added at last in the structure.**

```
typedef enum
{
    TASK_START_E = 0,
    TASK1_E= TASK_START_E,
    Task2_E,
    Task3_E,
    //Task4_E,
    //Task5_E,
    // Task6_E,
    // Task7_E,
    // Task8_E,
    // Task9_E,
    TOTAL_TASK,
} Task_En_t;
```

6. BUILD PROCEDURE

1. Open Project folder.

Name	Date modified	Type	Size
.vscode	30-07-2025 03:58 PM	File folder	
APP_TEST	30-07-2025 03:49 PM	File folder	
app_typedefs	30-07-2025 01:15 PM	File folder	
Build	30-07-2025 01:21 PM	File folder	
CORE	30-07-2025 01:15 PM	File folder	
Data_Acquire	30-07-2025 01:15 PM	File folder	
DataBank	30-07-2025 01:15 PM	File folder	
DefaultBuild	30-07-2025 01:20 PM	File folder	
DIAG	30-07-2025 01:15 PM	File folder	
HAL	30-07-2025 01:15 PM	File folder	
Lib	30-07-2025 01:16 PM	File folder	
PAL	30-07-2025 01:15 PM	File folder	
Setting	30-07-2025 01:21 PM	File folder	
src	30-07-2025 01:15 PM	File folder	
boot.asm	29-07-2025 10:28 AM	Assembler Source	7 KB
cstart.asm	29-07-2025 10:28 AM	Assembler Source	4 KB
iodef.h	29-07-2025 10:28 AM	C Header Source F...	1,192 KB
Project_build.bat	30-07-2025 12:57 PM	Windows Batch File	14 KB

2. Select Project_build.bat file and click enter.

```

W0561017:The evaluation period has expired

W0561320:Duplicate symbol "_main" in "D:\VCU_24pin\24PIN_SDK_V1\Lib\24Pin_Lib_v1.lib(r_cg_main)"
W0561320:Duplicate symbol "_main" in "D:\VCU_24pin\24PIN_SDK_V1\Lib\24Pin_Lib_v1.lib(main)"
W0561100:Cannot find "DEVICE_NAME.const" specified in option "start"

RAMDATA SECTION: 00002248 Byte(s)
ROMDATA SECTION: 00001dba Byte(s)
PROGRAM SECTION: 000165ec Byte(s)

Renesas Optimizing Linker Completed
Press any key to continue . . .

```

It is an indication that Renesas Optimizing Linker Completed.

Note: for Every Change in Application code, these two steps have to be followed.

6.1 How to find Hex file

1. Open the project file

Name	Date modified	Type	Size
.vscode	30-07-2025 03:58 PM	File folder	
APP_TEST	30-07-2025 03:49 PM	File folder	
app_typedefs	30-07-2025 01:15 PM	File folder	
Build	30-07-2025 01:21 PM	File folder	
CORE	30-07-2025 01:15 PM	File folder	
Data_Acquire	30-07-2025 01:15 PM	File folder	
DataBank	30-07-2025 01:15 PM	File folder	
DefaultBuild	30-07-2025 01:20 PM	File folder	
DIAG	30-07-2025 01:15 PM	File folder	
HAL	30-07-2025 01:15 PM	File folder	
Lib	30-07-2025 01:16 PM	File folder	
PAL	30-07-2025 01:15 PM	File folder	
Setting	30-07-2025 01:21 PM	File folder	
src	30-07-2025 01:15 PM	File folder	
boot.asm	29-07-2025 10:28 AM	Assembler Source	7 KB
cstart.asm	29-07-2025 10:28 AM	Assembler Source	4 KB
iodef.h	29-07-2025 10:28 AM	C Header Source F...	1,192 KB
Project_build.bat	30-07-2025 12:57 PM	Windows Batch File	14 KB

2. Open Build folder.

Name	Date modified	Type	Size
asm	30-07-2025 07:16 PM	File folder	
obj	30-07-2025 07:16 PM	File folder	
Output	30-07-2025 07:16 PM	File folder	

3. Open Output folder.
4. In output folder Hex file will available.

Name	Date modified	Type	Size
24PIN_SDK_V1.hex	30-07-2025 07:16 PM	HEX File	274 KB

5. For every build hex file gets updated.

7. CAN Baud Rate DID Configuration

A dedicated DID (0xF1FF) has been implemented to support CAN baud rate configuration. The configured CAN baud rate can be read using UDS service 0x22 (ReadDataByIdentifier) and written using UDS service 0x2E (WriteDataByIdentifier).

The configured baud rate shall be applicable for Reprogramming. During reprogramming, based on the application baud rate configuration, the PBL and SBL baud rates shall be updated accordingly. By default the CAN baud rate will be configured as 500 kbps.

The Sloki Flasher Tool version **2.1.8** has been updated to support controller baud rate configuration. Refer to the Flasher Tool User Manual at the link below for guidance on changing the baud rate.

Flasher tool manual link: [Sloki ECU Flasher User Manual.pdf](#)

7.1 Write CAN Baud Rate (UDS Service 0x2E)

The following request and response are used to write the CAN baud rate using UDS service 0x2E.

Physical Request	CAN ID	DLC	CAN frame Data Field							
			0	1	2	3	4	5	6	7
			DL	POS RES	DID	DID	Data Record	Data	Data	Data
	0x7F0	0x08	0x04	0x2E	0xF1	0xFF	XX	0x00	0x00	0x00

Physical Response	CAN ID	DLC	CAN frame Data Field							
			0	1	2	3	4	5	6	7
			DL	POS RES	DID	DID	Data	Data	Data	Data
	0x7F1	0x08	0x03	0x6E	0xF1	0xFF	0x00	0x00	0x00	0x00

The data record (**XX**) indicates the configured CAN baud rate.

0 - 500kbps,

1 - 125kbps,

2 - 250kbps,

3 - 1Mbps

➤ The following preconditions shall be met to update the CAN baud rate:

- For writing the baud rate (UDS service 0x2E), the ECU shall be in **Extended Diagnostic Session**.
- Security access shall be successfully unlocked prior to executing the write operation (0x2E).
- The newly configured baud rate shall be applied to the application after an ECU hard reset.

7.2 Read CAN Baud Rate (UDS Service 0x22)

The following request and response are used to read the CAN baud rate using UDS service 0x22.

Physical Response	CAN ID	DLC	CAN frame Data Field							
			0	1	2	3	4	5	6	7
			DL	SID	DID	DID	Data	Data	Data	Data
	0x7F0	0x08	0x03	0x22	0xF1	0xFF	0x00	0x00	0x00	0x00

Physical Response	CAN ID	DLC	CAN frame Data Field							
			0	1	2	3	4	5	6	7
			DL	POS RES	DID	DID	Data record	Data	Data	Data
	0x7F1	0x08	0x04	0x62	0xF1	0xFF	XX	0x00	0x00	0x00

The data record (**XX**) indicates the configured CAN baud rate.

- 0 - 500kbps,
- 1 - 125kbps,
- 2 - 250kbps,
- 3 - 1Mbps

8. NVM Read and Write Functions

8.1 nvm_status_En_t

This enumeration defines the status codes returned by NVM operations, indicating success, busy states, and various error conditions during nvm write operations.

Declaration

The nvm_status_En_t enumeration is declared in the **nvm_userinterface.h** file.

Enumeration Definition

```
typedef enum NVM_STATUS_T
{
    NVM_STATUS_OK           = 0,    /*Operation terminated successfully */
    NVM_STATUS_BUSY        = 1,    /*Operation is still ongoing */
    NVM_STATUS_SUSPENDED    = 2,    /*Flash operation is suspended */
    NVM_STATUS_ERR_CONFIGURATION = 3, /* The NVM configuration (descriptor) was wrong */
    NVM_STATUS_ERR_PARAMETER = 4,    /* A parameter of the NVM function call was wrong */
    NVM_STATUS_ERR_PROTECTION = 5,    /* Operation blocked due to wrong parameters */
    NVM_STATUS_ERR_REJECTED   = 6,    /*Flow error, e.g. another operation is still busy */
    NVM_STATUS_ERR_WRITE      = 7,    /*Flash write error */
    NVM_STATUS_ERR_ERASE      = 8,    /* Flash erase error */
    NVM_STATUS_ERR_BLANKCHECK = 9,    /*Flash blank check error */
    NVM_STATUS_ERR_COMMAND    = 10,   /*Unknown command */
    NVM_STATUS_ERR_ECC_SED     = 11,   /* Single bit error detected by ECC */
    NVM_STATUS_ERR_ECC_DED     = 12,   /* Double bit error detected by ECC */
    NVM_STATUS_ERR_INTERNAL    = 13,   /*Library internal error */
    NVM_STATUS_CANCELLED      = 14    /*Flash operation is cancelled */
}nvm_status_En_t;
```

8.2 NVM Write Function

Function prototype:

```
nvm_status_En_t NvmWriteBlock(uint32_t block_id_u32, uint8_t *buffer_pu8, uint32_t  
size_u32)
```

Parameters:

block_id_u32 : Logical block ID where data needs to be written
buffer_pu8 : Pointer to the data buffer to be written
size_u32 : Number of bytes to write (typically 64 bytes per block)

Return Value:

- NVM_STATUS_OK: Write operation completed successfully
- NVM_STATUS_ERROR: Write operation failed
- NVM_STATUS_BUSY : NVM is busy (if applicable)

Description:

This function writes data to Non-Volatile Memory (NVM) at a specified block location.

NOTE: Ensure size_u32 does not exceed block size (typically 64 bytes per block)

Example:

```
#include "nvm_userinterface.h"  
  
void App_NvmWriteExample(void)  
{  
    uint8_t writeData[10] = {0xA0, 0x50, 0xAA, 0x55, 0x11, 0x22, 0x33, 0x44,  
0x55, 0x66};  
  
    NvmWriteBlock(0U, writeData, 64U);  
}
```

8.3 NVM Read Function

Function prototype:

```
uint8_t NvmReadBlock(uint32_t block_id_u32, uint8_t *buffer_pu8, uint32_t size_u32)
```

Parameters:

block_id_u32 : Logical block ID where data needs to be read
buffer_pu8 : Pointer to the data buffer where read data will be stored
size_u32 : Number of bytes to write (typically 64 bytes per block)

Return type: uint8_t (8bit)

Description:

This function is used to read data from Non-Volatile Memory (NVM) from a specified block location.

NOTE: Ensure size_u32 does not exceed block size (typically 64 bytes per block)

Example:

```
#include "nvm_userinterface.h"

void App_NvmReadExample(void)
{
    uint8_t readData[10]={0};

    NvmReadBlock(0U, readData, 64U);
}
```

Caution: The NVM is a emulated EEPROM. The entire block(64 bytes) will be erased before updating the data. Hence entire block data must be backed up before calling “NvmWriteBlock” to avoid the data loss.

THANK YOU